

Ingestion Pipeline Design for Real Estate AI Platform

Objective

The objective of the ingestion pipeline is to transform heterogeneous data sources into a retrieval-optimized representation that enables accurate, scalable, and low-latency retrieval while maintaining tenant isolation, supporting incremental updates, and preserving document hierarchy.

The design should optimize for retrieval quality rather than simply storing files.

Types of Data

The platform receives multiple forms of data:

Structured Data

Examples:

- Property listings
- Price history
- Latitude and longitude
- Agent information
- CRM records
- Property metadata

These data are naturally queryable using SQL and therefore do not necessarily require semantic embeddings.

Semi-Structured Data

Examples:

- XML
- JSON
- API payloads

These contain both metadata and textual content.

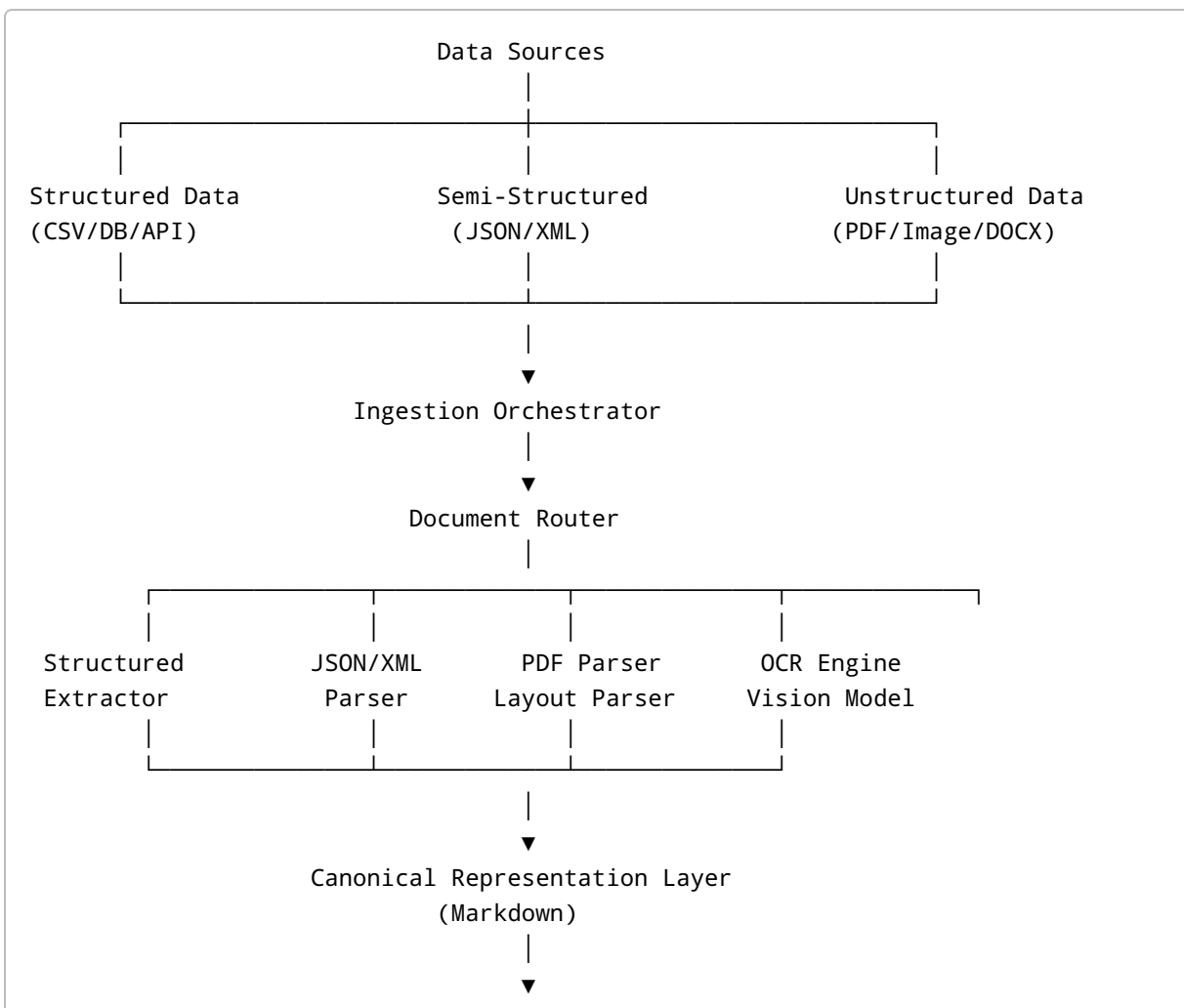
Unstructured Data

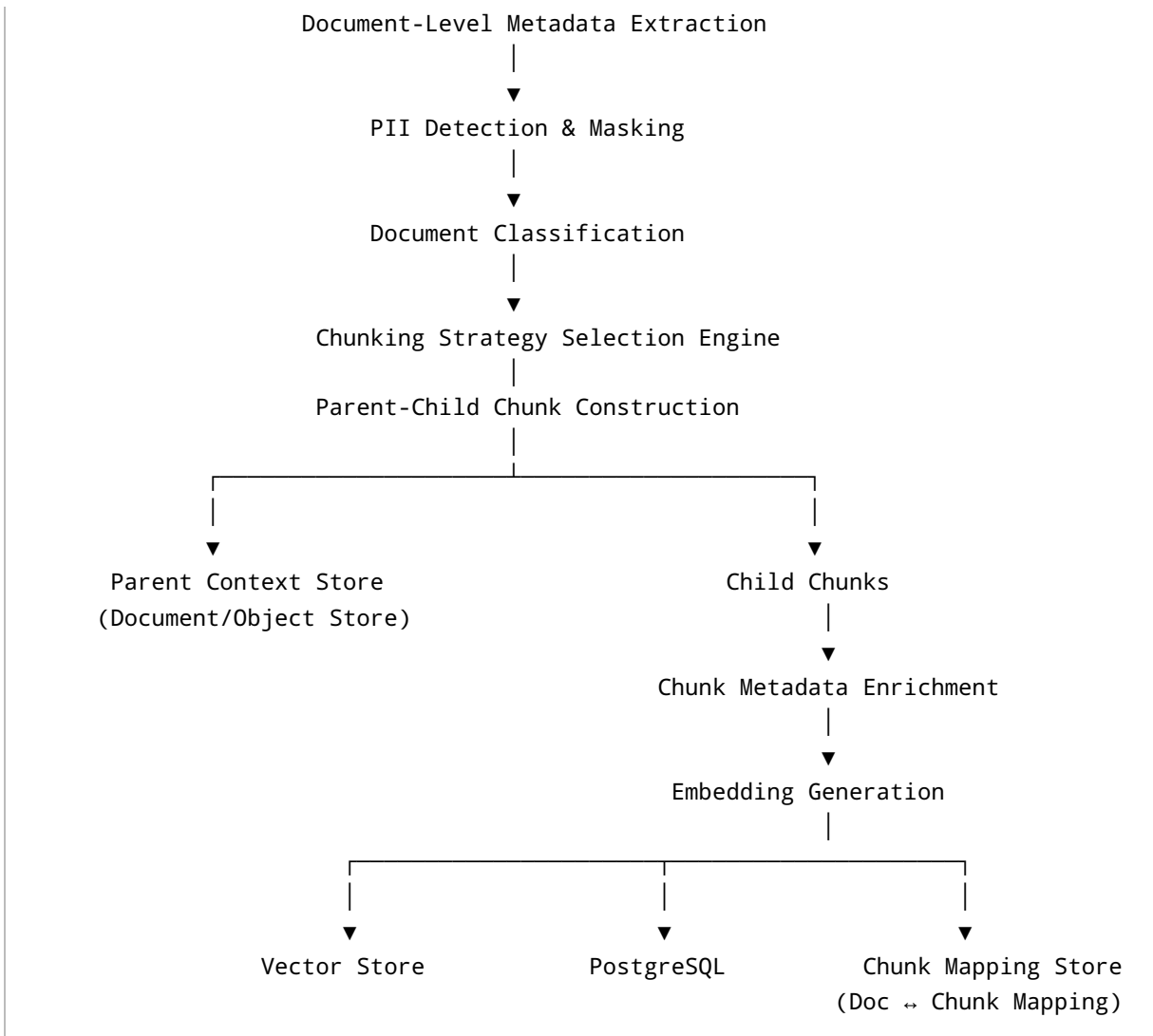
Examples:

- Sale deeds
- Title deeds
- Legal contracts
- Property brochures
- Emails
- Images
- Floor plans
- Scanned PDFs

These require parsing and semantic indexing.

High-Level Architecture





Ingestion Orchestrator

The orchestrator controls the entire ingestion workflow.

Responsibilities:

- Event-driven ingestion
- Retry handling
- Parallel processing
- Version control
- Idempotency

Typical implementation:

- Pub/Sub
- Kafka
- Workflows
- Airflow

This enables scalable asynchronous ingestion.

Document Router

Different document types require different processing pipelines.

Examples:

Document Type	Route To
CSV	Structured Extractor
JSON/XML	Parser
PDF	Layout Parser
Scanned PDF	OCR
Images	Vision Model
Floor Plans	Vision Language Model

Routing prevents forcing every document through the same parser.

Canonical Representation Layer

After extraction, every document is transformed into Markdown.

Example:

```
# Ownership Transfer

## Eligibility

Property can be transferred to family members.

## Restrictions
```

Transfer requires approval.

Why Markdown?

- Human-readable
- Preserves hierarchy
- LLM friendly
- Easy chunking
- Common representation across formats

The markdown file is stored permanently.

It becomes the canonical source for reprocessing.

Metadata Extraction

Before chunking, document-level metadata is extracted.

Examples:

```
{
  "tenant_id": "tenant_1",
  "document_id": "doc_123",
  "document_type": "sale_deed",
  "version": "v7",
  "property_id": "P100"
}
```

This metadata applies to the entire document.

Document-level metadata should be created before chunking.

PII Detection and Masking

Sensitive information should not be embedded.

Examples:

- Aadhaar number
- PAN number

- Email address
- Phone number
- Bank account

Masking example:

9876543210

↓

98XXXX3210

This prevents sensitive information from leaking into vector embeddings.

Document Classification

Different document types require different chunking strategies.

Examples:

- Sale Deed
- Title Deed
- Brochure
- CRM Notes
- Email
- Property Description

Classification determines how chunking will be performed.

Chunking Strategy

There is no universal chunking strategy.

Different documents require different approaches.

Hierarchical Chunking

Suitable for:

- Sale deeds
- Legal documents
- Title deeds

Structure:

Document → Section → Subsection → Child chunks

This preserves hierarchy.

Layout-Aware Chunking

Suitable for:

- Brochures
- Floor plans

Image and accompanying text remain together.

This prevents losing context.

Semantic Chunking

Suitable for:

- CRM notes
- Emails

Chunk boundaries are determined by semantic similarity.

Fixed-Size Chunking

Used as fallback.

Not preferred because it may break context.

Parent-Child Architecture

The system creates two types of nodes.

Parent Nodes

Contain:

- Section text
- Subsection text
- Summaries (optional)

Stored in:

- Document store
- Object store

Parent nodes are not embedded.

Child Chunks

Contain smaller pieces of information.

Child chunks are embedded.

Each child contains:

```
{
  "chunk_id": "c10",
  "parent_id": "p3",
  "document_id": "doc_123",
  "tenant_id": "tenant_1"
}
```

This enables context expansion during retrieval.

Chunk Metadata Enrichment

Chunk-specific metadata is generated after chunking.

Examples:

```
{
  "chunk_id": "c10",
  "parent_id": "p3",
```



```
"section": "Ownership",  
"subsection": "Transfer",  
"version": "v7"  
}
```

Metadata inheritance occurs from parent to child.

Document-level metadata is created before chunking.

Chunk-level metadata is created after chunking.

Document-to-Chunk Mapping Store

An additional mapping layer maintains relationships between documents and chunks.

Example:

document_id	chunk_id	parent_id
doc_123	c1	p1
doc_123	c2	p1
doc_123	c3	p2

This mapping enables:

- Incremental updates
- Fast deletions
- Version management
- Traceability
- Parent-child retrieval

Without this layer, updating vectors becomes expensive.

Embedding Generation

Only child chunks are embedded.

Requirements:

- Multilingual support
- Cross-lingual support

- Quantization support
- Cost optimization

Recommended:

Gemini Embedding 001

Benefits:

- Adjustable dimensions
 - Good multilingual support
 - Cross-lingual retrieval
 - MRL support
-

Storage Strategy

PostgreSQL

Stores structured attributes.

Examples:

- Price
- Latitude
- Longitude
- Bedrooms
- Agent information

Supports exact filtering.

Vector Store

Stores:

- Chunk embeddings
- Chunk metadata

Recommended:

Qdrant

Benefits:

- HNSW
 - Strong metadata filtering
 - Hybrid retrieval
 - Quantization support
-

Parent Context Store

Stores:

- Section text
- Subsection text
- Parent summaries

Supports context expansion.

Mapping Store

Stores:

Document ↔ Chunk relationships.

Supports incremental indexing.

Incremental Update Strategy

When a document changes:

Avoid:

Re-indexing entire corpus.

Instead:

Document Changed ↓ Lookup Mapping Store ↓ Identify affected chunks ↓ Re-chunk modified sections ↓
Generate embeddings ↓ Upsert vectors ↓ Update mapping

This minimizes cost.

Multi-Tenancy

Every object contains:

- tenant_id
- document_id
- version

Growth strategy:

Shared Collection ↓

Tenant Namespace ↓

Partition Per Tenant ↓

Dedicated Collection ↓

Dedicated Infrastructure

This balances cost and scalability.

Observability

Monitor:

Ingestion

- Documents/hour
- Parser failures

Chunking

- Average chunk size
- Chunk distribution

Embedding

- Embedding latency
- Cost

Indexing

- Vector count

- Update latency

Data Quality

- OCR accuracy
- Metadata completeness

Freshness

- Index lag
- Version mismatch

Final Recommendation

I would design the ingestion pipeline around retrieval quality rather than file formats.

Documents are routed to specialized parsers and transformed into a canonical markdown representation. Document-level metadata is extracted before chunking, followed by PII masking and document classification. A document-specific chunking strategy generates parent and child nodes. Parent nodes are stored in a document store, while only child chunks are embedded and stored in the vector database. A dedicated mapping layer maintains relationships between documents and chunk IDs, enabling incremental updates, efficient deletions, traceability, and version management. Structured attributes remain in PostgreSQL, and every component carries tenant-aware metadata to support scalable multi-tenant deployments.